

AI 播客生成系统汇报

1. 项目整体介绍

这个项目是一个 AI 播客生成与个性化推荐系统。

我们之所以想做这个项目，是因为传统播客的生产流程比较繁杂。从选题、资料收集、内容整理、脚本撰写，到录音、剪辑和发布，每个环节都需要投入大量人力，而且生产周期较长，很难快速响应热点信息。

在 AI 时代，我们认为信息的生产和传播可以被进一步自动化。AI 不只是提高效率，也可以帮助我们完成内容筛选、主题整合、脚本生成和个性化分发推荐。如果流程设计得足够好，自动生成的内容也可以具备一定深度，而不只是简单的信息搬运。

核心目标是：把分散在 RSS 信息源里的新闻内容，自动整理成可以收听的播客节目，并根据用户的播放、点赞、收藏、跳过等行为，给用户推荐更符合兴趣的播客。

整个系统可以理解为一条完整链路：

RSS 新闻进入系统后，先经过抓取、清洗、去重和聚类，系统会把同一事件或相近主题的新闻合并成一个新闻组。

然后 AI 根据这个新闻组生成播客脚本，再通过语音合成生成音频。

用户收听之后产生行为数据，推荐系统再利用这些行为数据调整后续推荐结果。

即从新闻采集，到 AI 生成播客，再到用户个性化推荐，形成一个完整的内容生产和内容分发闭环。

2. 系统整体流程

项目整体可以分为五个阶段：

新闻采集

系统从多个公开的 RSS 信息源抓取新闻内容。

这些 RSS 源可以是系统预设的，也可以是用户自定义的。抓取完成后，系统会把不同来源的新闻统一成相同的数据格式，方便后续处理。

新闻整理

系统会对新闻做清洗、去重，并基于 embedding 计算相似度后进行聚类。

清洗是去掉无用内容，去重是避免同一条新闻重复生成，聚类是把同一事件或相近主题的新闻放到同一个组里，减少同一事件被多个播客报道。

脚本生成

系统把整理好的新闻组交给大模型，让大模型生成结构化播客脚本。

这个脚本不是简单复述新闻，而是会按照播客节目形式组织内容，包括开场、主体讲解、过渡和结尾。

音频合成

系统把生成好的脚本交给语音合成服务，生成每一段语音，最后把这些语音片段合并成完整的播客音频。

入库和推荐

3. 核心生成流程

核心生成流程主要解决一个问题：怎样把一批新闻自动变成一集可以播放的播客。

3.1 抓取 RSS 新闻

系统首先读取 RSS 源配置，然后从这些信息源中抓取最新内容。

不同 RSS 源返回的数据格式可能不完全一致，所以系统会把它们统一整理成标准新闻数据。每条新闻通常包括标题、摘要、来源、发布时间和原文链接等信息。

这一阶段的作用是提供原始素材。

3.2 新闻去重

新闻抓取后，系统会先做去重。

历史去重：系统会记录之前已经用于生成播客的新闻。如果某条新闻已经被使用过，下一次生成时就会自动跳过。

这样可以避免重复生产相同播客，也能让系统每次尽量生成新内容。

3.3 新闻聚类

去重之后，系统会把新闻进行聚类。

聚类的目的不是简单分类，而是判断哪些新闻属于同一个事件或同一个主题。

4. 向量聚类流程

聚类流程是这个项目里比较关键的一部分。

它决定了系统能不能把分散的新闻整理成适合生成播客的主题组。

4.1 为什么需要聚类

如果不做聚类，系统可能会遇到两个问题。

第一个问题是内容太碎。一条新闻的信息量有限，直接生成播客可能很短，也缺少背景和完整性。

第二个问题是内容重复。多个 RSS 源可能报道同一个事件，如果每条都生成一集播客，就会出现多集内容高度相似的节目。

聚类就是为了解决这两个问题。

它会把相似新闻合并，让一集播客围绕一个主题或事件展开。

4.2 聚类前的数据准备

在聚类之前，系统会先整理新闻内容。

主要整理的信息包括：

- 新闻标题

新闻摘要

5. 内容生产

聚类完成后，系统并不是直接把新闻简单拼接成播客，而是进入内容生产阶段。

我们迭代了很多版本提示词。因为大模型生成内容的质量，很大程度上取决于提示词怎么设计。如果提示词只要求“总结新闻”，生成结果就会像普通新闻摘要。

我们的提示词明确要求节目结构、主持人口吻、信息层次和听众体验，生成结果才更符合真正的播客。

在提示词设计上，我们把模型设定为真正的内容制作人，先理解新闻事实，再梳理背景、设计节目结构、控制对话节奏，最后生成适合收听的播客脚本。

播客采用双角色对话形式。

A 是新闻主播，主要负责立题、提问、转场、总结和把复杂信息翻译成更好懂的话；

B 是分析师，主要负责补充事实、解释背景和展开分析。这样设计是为了避免一个角色一直讲到底，让节目更像真实播客中的主持人与嘉宾交流。

在播客内容组织上，我们主要关注五个点：

6. 混合推荐系统

推荐系统主要解决的问题是：用户打开播客列表时，系统应该优先推荐哪些内容。

它不是只按发布时间排序，也不是只按热度排序，而是综合用户行为、内容相似度、全站热度、新鲜度、近期兴趣和完播情况进行混合推荐。

6.1 总体流程

推荐流程可以分成五步：

1. **读取用户历史行为**：系统会收集用户播放、点赞、收藏、完播、点击、继续播放、暂停、跳过等行为。
2. **把行为转换成兴趣强度**：不同行为代表不同程度的兴趣。比如收藏通常比播放更能说明用户喜欢，刚开始就跳过则说明用户可能不感兴趣。
3. **计算多个推荐分数**：系统会分别计算热度分、协同过滤分、内容相似分、新鲜度分、近期序列分和完播分。
4. **根据用户阶段选择推荐策略**：新用户、少量行为用户和行为较多用户使用不同的策略。
5. **加权融合得到最终排序**：把多个分数融合后输出结果，并返回推荐理由。

6.2 用户行为权重

推荐系统会把用户行为分成正反馈、中性反馈和负反馈。

正反馈包括：

7. 数据闭环

这个项目最重要的特点是形成了数据闭环。

一开始，系统从 RSS 源抓取新闻，生成播客。

播客上线后，用户会产生播放、点赞、收藏、跳过等行为。

这些行为又会进入推荐系统，帮助系统了解用户兴趣。

推荐系统再把更合适的播客推给用户，用户继续产生新的行为。

整个链路可以概括为：

新闻进入系统，AI 生成播客，用户收听播客，用户行为反馈给推荐系统，推荐系统再影响后续内容分发。

这就是从内容生产到个性化推荐的闭环。

8. 开源软件相关

最后从开源项目的角度说明一下这个项目。

这个项目不是只关注功能能不能跑通，也按照开源项目的方式来组织。项目中包含了比较完整的说明文档、贡献指南、安全说明和开源许可证，方便其他人理解、运行、修改和继续扩展。

首先，在许可证方面，项目采用 Apache License 2.0。这个许可证允许他人在遵守许可证要求的前提下使用、修改和分发项目代码，方便项目传播，也比较适合后续继续扩展成更完整的应用。

其次，在协作方式上，项目给贡献者提供了贡献指南。贡献者在修改前可以先阅读项目说明，了解前后端结构和运行方式。

第三，在安全方面，项目也包含安全说明。如有人发现安全问题，我们不建议直接公开暴露，而是通过私下方式报告，先修复再公开说明。

最后，从开源价值来看，后续其他开发者可以在这个基础上继续扩展，例如支持更多语音服务、优化推荐策略，或者把它部署成面向真实用户的播客平台。

9. 总结

在过去，信息的分发主要依赖编辑选择；现在，推荐算法已经在很大程度上承担了内容分发的工作。

我们认为在 AI 时代，信息会被自动生产出来，而人不再只是内容的直接生产者，更会成为流程的设计者、质量的把关者和价值方向的定义者。